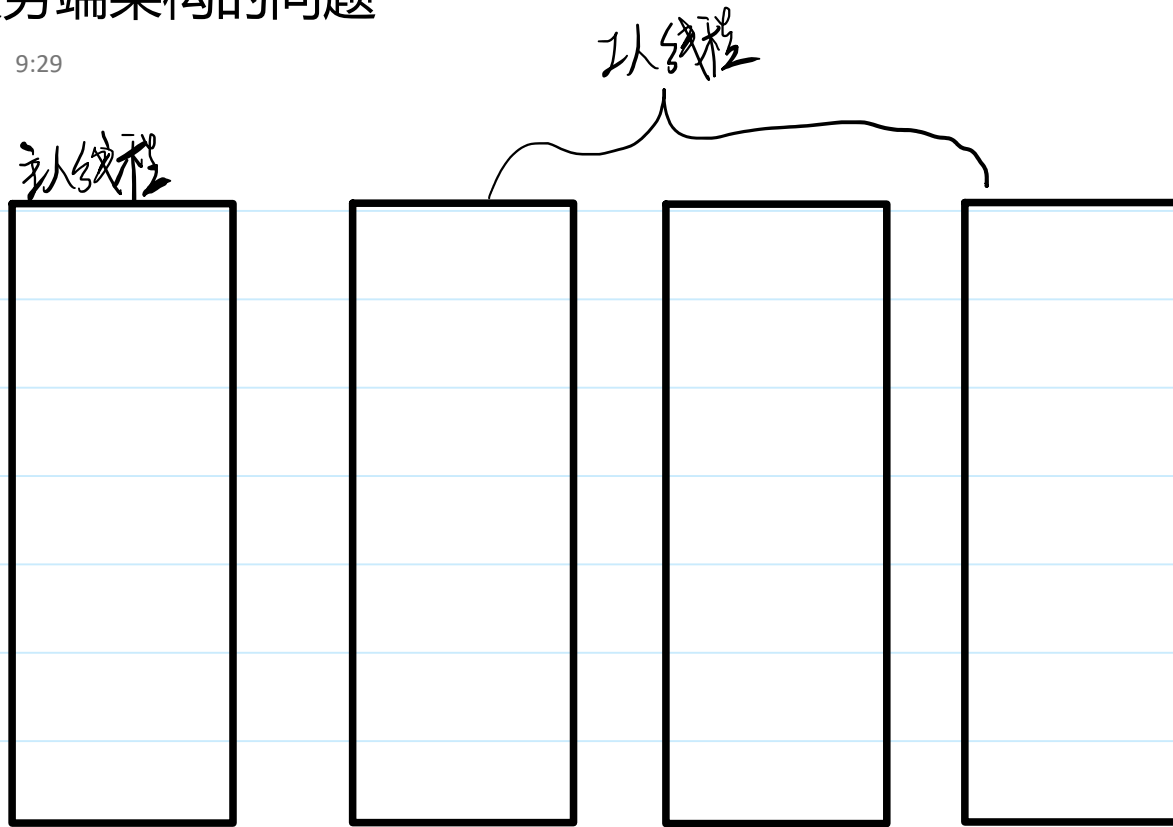


现有的服务端架构的问题

2025年6月13日

9:29



主人的职责：

- 1、创建&&管理工人
- 2、用epoll监听sockfd，accept得到netfd交给工人

工人的职责：

- 1、执行业务，处理netfd

→ 饥饿.

饥饿

2025年6月13日 9:32

有多个客户端到来的时候，有些客户端需要等待很长的时间



长连接

服务端的工人一旦拿到连接，就会执行多条指令，才退出 → 饥饿

短连接

服务端的工人拿到netfd，就执行一条指令，直接
close

短连接模式的优势和代价

2025年6月13日 9:38

```
while(1){  
    connect  
    send  
    close  
}
```

 第一次send完到第二send之间，不用占用工人

1、服务端的代码变复杂了

一个客户端执行多条指令，不是同一个工人处理的 ---> 身份认证

2、服务端的平均性能变差了

频繁地建立连接和断开连接

身份认证

2025年6月13日 9:43

session 方案

在服务端弄一个共享数据结构，记录了每个有效的用户

token 方案

优势：服务端是无状态的，支持水平扩展



token的生成方案

2025年6月13日 9:57

1、自己设计 md5、sha1、sha256

2、现成的token方案

JWT

JSON Web Tokens

 https://www.ruanyifeng.com/blog/2018/07/json_web_token-tutorial.html

l8w8jwt

2025年6月13日 9:59

[←](#) [↺](#) <https://github.com/GlitchedPolygons/l8w8jwt>

[📁 考研机试](#) [📁 GO](#) [📁 C++&OS](#) [📁 others](#) [📁 CMU](#) [🔗 C++ Insights](#) [🐍 py ref](#) [🌐 Deep Learning Syst...](#) [🌐 libevent](#) [📁 2023考研机试](#) [🔗 记一次解决VMWare...](#) [🇺🇸](#)

[☰](#)  GlitchedPolygons / l8w8jwt

[🔍 Type to search](#)

[<> Code](#) [🔔 Issues 2](#) [🔗 Pull requests](#) [🎬 Actions](#) [📁 Projects](#) [🛡 Security](#) [📊 Insights](#)

 **l8w8jwt** Public

[❤ Sponsor](#) [👁 Watch 4](#)

[🔗 master](#) [🔗 1 Branch](#) [🏷 20 Tags](#)

[🔍 Go to file](#) [t](#) [Add file](#) [<> Code](#)

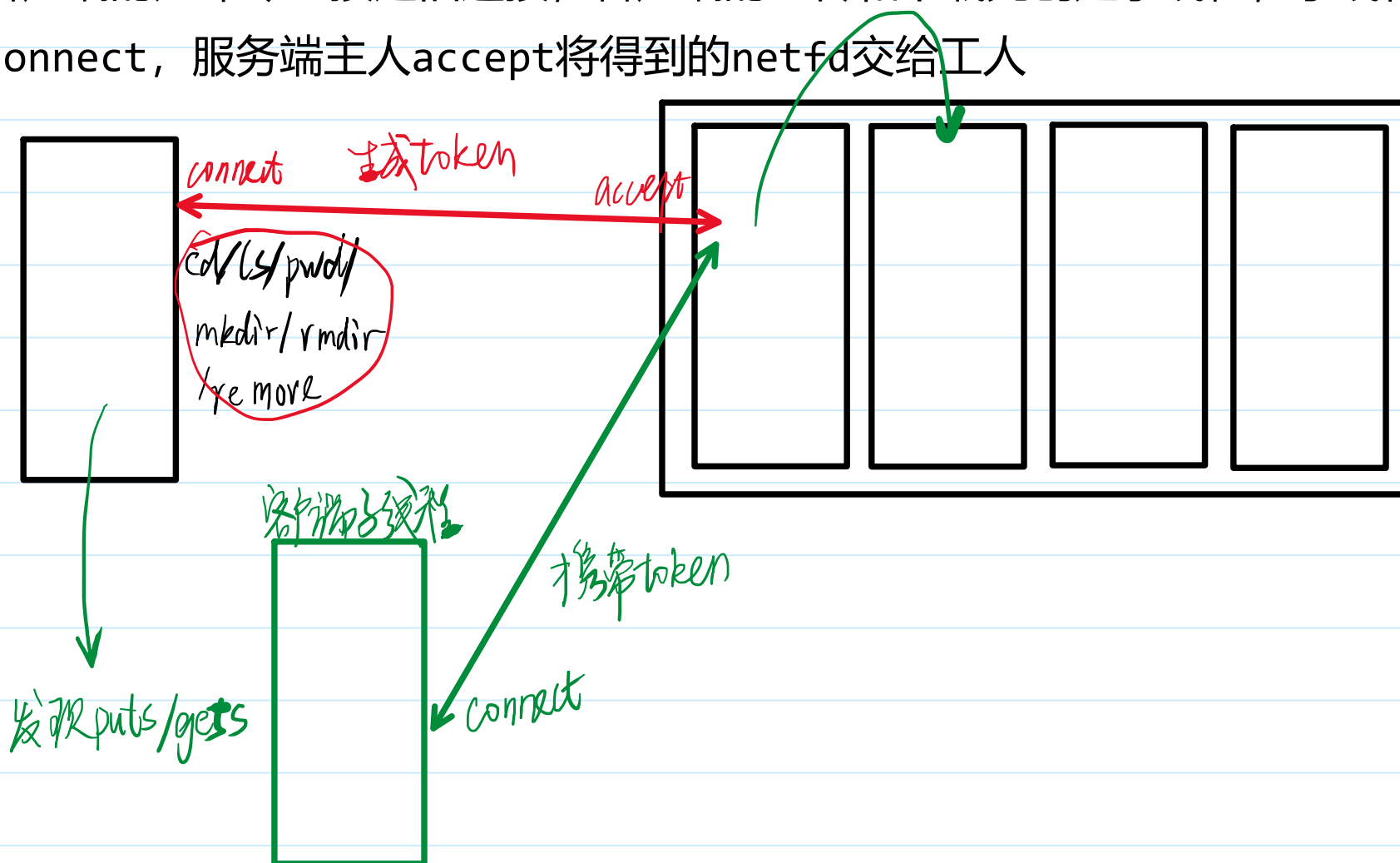
 **GlitchedPolygons** Recompiled docs. ✓

ea57a66 · 3 months ago ⌚ 313 Commits

改造主人线程 ---> 长短命令分离

2025年6月13日 10:03

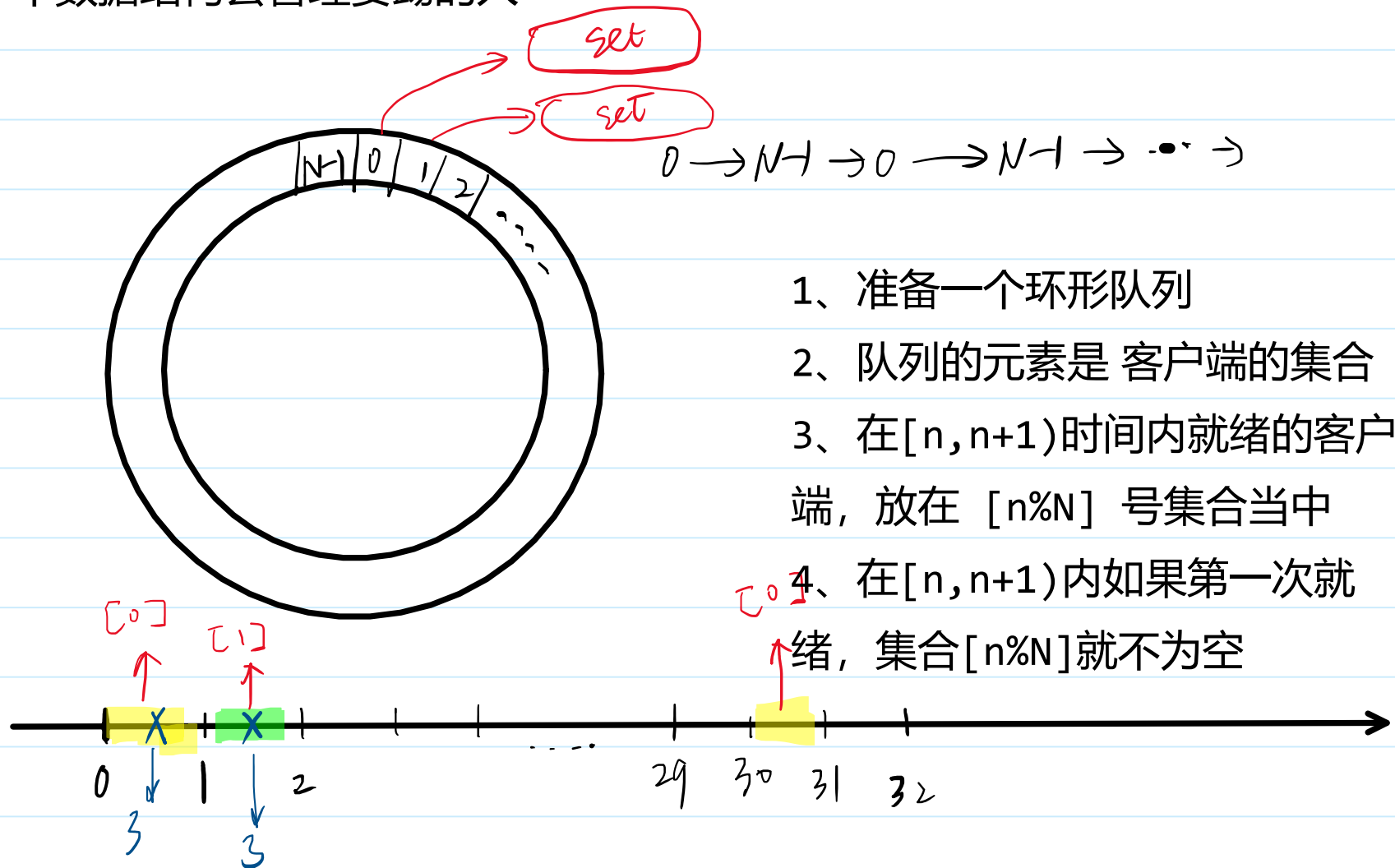
主人线程现在accept得到netfd之后，不交给工人，用epoll去监听netfd
客户端的短命令直接走旧连接，客户端的上传和下载先创建子线程，子线程去connect，服务端主人accept将得到的netfd交给工人



超时退出

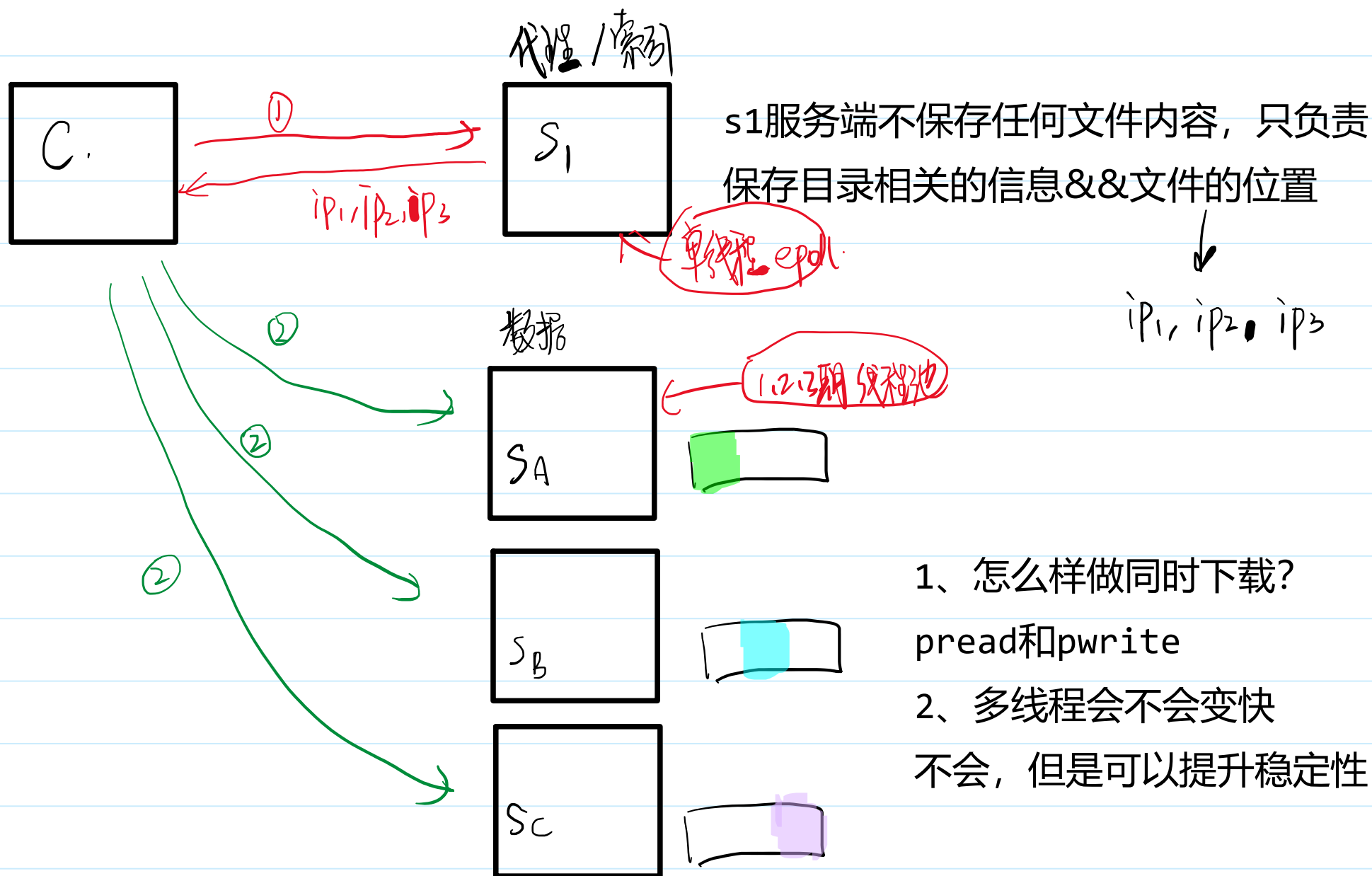
2025年6月13日 10:26

如果一个客户端30s不发消息，就踢掉
设计一个数据结构去管理要踢的人



第五期 1 多点下载

2025年6月13日 10:56



多线程同时写入用pread和pwrite

2025年6月13日 11:01

```
ssize_t pread(int fd, void *buf, size_t count, off_t offset);  
ssize_t pwrite(int fd, const void *buf, size_t count, off_t offset);
```

2 支持FTP协议

2025年6月13日 11:07

同时支持 TLV协议和FTP协议

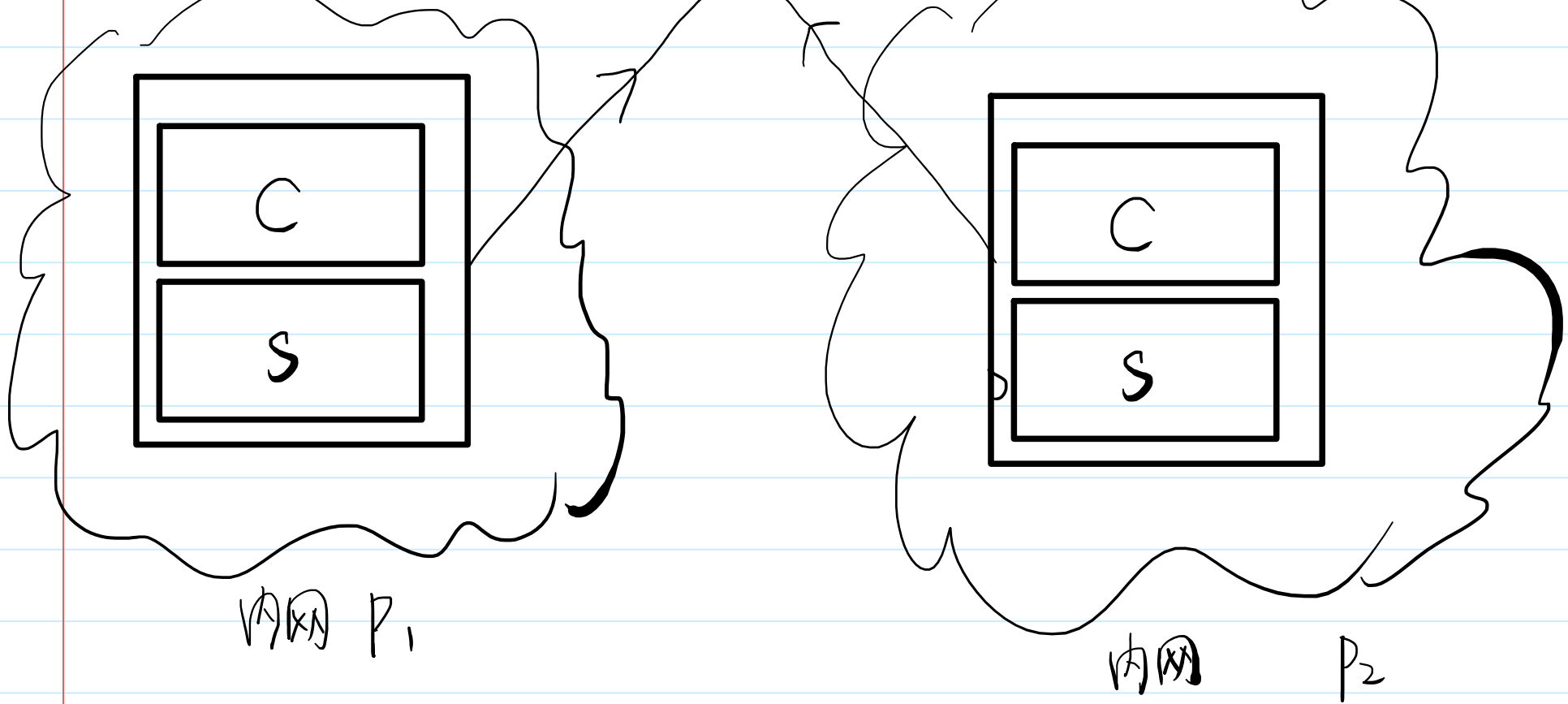
想研究实际FTP协议，详见

<http://blog.csdn.net/yxyhack/article/details/1826256>

3 P2P

2025年6月13日 11:08

穿透服



STUN协议